

Planificación de tareas

- Introducción
- Modelo simplificado
- Modelo simplificado con tareas esporádicas
- Interacción entre tareas. Inversión de prioridad
- Planificación en POSIX

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

1

Planificación de tareas

- **Objetivo:**
 - Planificar el uso de los recursos para conseguir cumplir especificaciones temporales
- **Planificación de tareas:**
 - Algoritmo de planificación para repartir los recursos de procesamiento
 - Método de análisis para verificar si se cumplen las restricciones
 - Es necesario considerar el peor caso posible
 - El problema **no tiene por qué tener solución**

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

2

Diferentes posibilidades

- **Sistemas de tiempo compartido (UNIX, Linux):**
 - Objetivo: repartir equitativamente los recursos
 - Prioridades dinámicas, en función de un algoritmo no trivial
 - El tiempo de respuesta no es lo más importante
 - Las decisiones pueden tomarse según un periodo básico ("tick")
- **Sistemas en tiempo real típicos: Prioridad fija con expulsión ("fixed priority and preemptive")**
 - Prioridades fijas con significado claro
 - Idealmente la tarea más prioritaria debería ejecutarse tan pronto como esté lista ("ready")
 - Las tareas de baja prioridad son expulsadas de la CPU
 - Diseñados para conseguir tiempos de respuesta predecibles
- **Otros enfoques:**
 - Ejecutivo cíclico (no es programación concurrente; ver tema 3)
 - Planificación cooperativa (la CPU debe cederse voluntariamente)
 - Prioridad al plazo más cercano (EDF, "early deadline first")

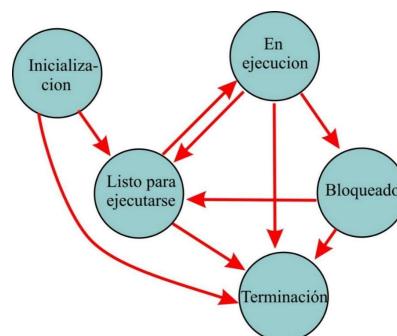
02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

3

Prioridades fijas y desalojo

- Las tareas se "despachan" por orden de prioridad
- Las tareas menos prioritarias son expulsadas de la CPU si es necesario
- Recordemos el diagrama de estados de una tarea (proceso, hilo, "task"...):



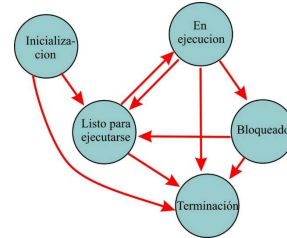
02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

4

Prioridades fijas y desalojo

- Principales transiciones de estado:
 - Las tareas en ejecución llegan a un punto de bloqueo y liberan la CPU
 - Se selecciona a tareas listas para ejecución para que continúen
 - Las tareas bloqueadas terminan desbloqueándose y transformándose en listas para ejecución
 - Las tareas en ejecución pueden ser “expulsadas” y pasar a listas para ejecución
- Planificación (“scheduling”) tomar decisiones sobre el estado de las tareas
- Las decisiones se toman indirectamente, fijando prioridades



02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

5

Prioridades fijas y desalojo

- Hablamos de programación concurrente, no de un ejecutivo cíclico:
 - Tareas especificadas como programas separados
 - En cualquier momento puede producirse una conmutación de tareas no programada
 - Existe el concepto de prioridad
 - El reparto del tiempo se realiza de manera indirecta a través de la prioridad, no directa y por programa

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

6

Restricciones temporales

- **Ciclo (T)**
 - No existe en actividades esporádicas (activadas por eventos)
- **Coste (C)**
 - Tiempo de ejecución **máximo** (peor) si no existen interferencias de otras actividades
 - Difícil de medir con precisión (¿Cuál es el peor de los casos que a la vez sea realista?)
- **Tiempo de respuesta (R)**
 - Tiempo de ejecución **máximo** (peor) en condiciones reales de funcionamiento
 - Nunca inferior al coste (es imposible...)
- **Tiempo límite, plazo o “deadline” (D)**
 - Máximo tiempo de respuesta R admisible (condición de diseño)
 - Nunca puede fijarse inferior al coste, C
- **Restricción temporal: $R \leq D$**

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

7

Modelo simplificado

- **Suposiciones:**
 - El conjunto de tareas es estático (las tareas no se crean ni desaparecen durante la ejecución del sistema)
 - Todas las tareas son periódicas
 - Las tareas son independientes entre sí. No interaccionan salvo para pedir CPU
 - Los plazos (“deadlines”) son iguales a los periodos en todas las tareas
 - Se considera despreciable el coste de la conmutación de tareas
- **Asignación **óptima** de prioridades: **Monótona en frecuencia** (“rate monotonic scheduling”)**
 - Cuanto menor periodo, mayor prioridad
 - ¿En qué sentido es óptima? **Si el problema tiene alguna solución, ésta es una de ellas**

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

8

Factor de utilización

- Factor de utilización:

$$U = \sum_{i=1}^N \frac{C_i}{T_i}$$

- Medida de la carga del procesador
- Si sólo hay un procesador, debe cumplirse $U \leq 1$

- Condición de garantía de plazos (**modelo simplificado**):

- Si la prioridad se asigna de manera óptima (monótona en frecuencia) los plazos se cumplen si

$$U = \sum_{i=1}^N \frac{C_i}{T_i} \leq U_0(N) = N(2^{1/N} - 1)$$

- Donde N es el número de tareas, y $U_0(N)$ es la **utilización mínima garantizada**, función del número de tareas del sistema.
- Para $N \rightarrow \infty$, $U_0(N) \rightarrow 0,693$
- Condición **suficiente, no necesaria** (es posible que funcione sin cumplirse - aunque sí es cierto que debe ser $U \leq 1$ con un solo procesador)

N	$U_0(N)$
1	1,000
2	0,828
3	0,779
4	0,756

02/12/2025

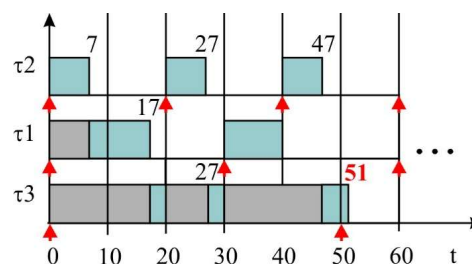
© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

9

Factor de utilización: Ejemplo

- Ejemplo:

Task	T	C	P	C/T	R
τ_1	30	10	2	0,333	17
τ_2	20	7	3	0,35	7
τ_3	50	10	1	0,2	51



$$U_0(3) = 3(2^{1/3} - 1) = 0,779$$

$$U = \sum_{i=1}^N \frac{C_i}{T_i} = 0,333 + 0,35 + 0,2 = 0,8833 > U_0(3)$$

- No hay ninguna conclusión con U . $U < 1$; podría funcionar
- Se comprueba dibujando el cronograma que en este caso falla (pero **a priori podría haber funcionado...**)

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

10

Instante crítico

- Si no se verifica la condición, hay que comprobar si se cumplen o no los tiempos
- Simular funcionamiento (dibujar cronograma)
- ¿Hasta cuando hemos de dibujar el cronograma para decidir que funciona?
 - Valdría un **hiperperiodo**, intervalo tras el cual se repite el funcionamiento de todas las tareas (como en un ejecutivo cíclico)
 - Mínimo común múltiplo de periodos; $O(N^N)$ en el peor de los casos (puede que sea un número muy grande...)
- Otra opción: **Primer ciclo desde el instante crítico**
 - **Instante crítico**: Todas las tareas se activan a la vez
 - El primer ciclo de cada tarea después del instante crítico es el peor de los casos: **Tiempo de respuesta máximo**

02/12/2025

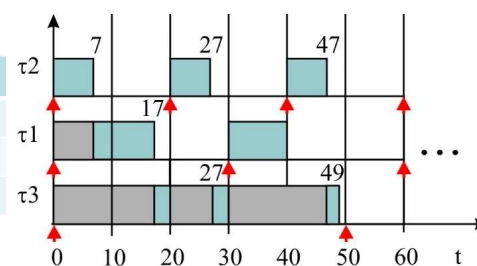
© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

11

Ejemplo

- Ejemplo:

Task	T	C	P	C/T	R
τ_1	30	10	2	0,333	17
τ_2	20	7	3	0,35	7
τ_3	50	8	1	0,16	49



$$U_0(3) = 3(2^{1/3} - 1) = 0,779$$

$$U = \sum_{i=1}^N \frac{C_i}{T_i} = 0,333 + 0,35 + 0,16 = 0,8433 > U_0(3)$$

- No hay ninguna conclusión con el criterio
- Se comprueba que en este caso **se cumplen los plazos**
- Basta comprobar el primer ciclo tras el instante crítico

02/12/2025

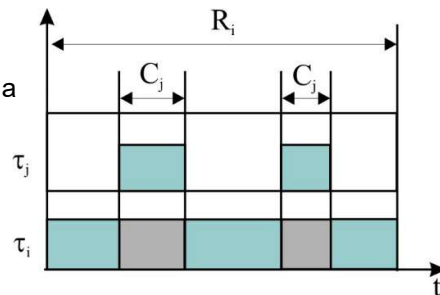
© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

12

Cálculo del tiempo de respuesta

- El factor de utilización no siempre ayuda, ni se puede generalizar a casos más complejos
- Objetivo:
 - Calcular tiempo de respuesta del primer ciclo de cada tarea tras el instante crítico (el peor de los casos posibles)
 - Realizar el cálculo mediante un algoritmo, no dibujando cronogramas
- Tiempo de respuesta:
 - Coste propio más interferencia de tareas más prioritarias

$$R_i = C_i + I_i$$



02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

13

Cálculo de la interferencia máxima

- Es la que sucede a partir del instante crítico
 - Calcular número de veces que se ejecuta cada tarea más prioritaria durante R_i
 - Multiplicar por el coste de cada tarea más prioritaria:

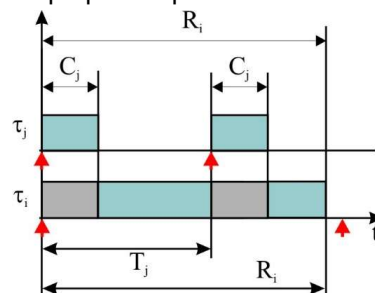
$$I_i^j = \left\lceil \frac{R_i}{T_j} \right\rceil C_j$$

- Redondeo al alza: Porque un solape parcial provoca una interferencia completa

- Interferencia total:

$$R_i = C_i + \sum_{j \in hp(i)} I_i^j$$

$$R_i = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j$$



02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

14

Ecuación del tiempo de respuesta

- El tiempo de respuesta R_i es la menor solución de la ecuación

$$w_i = C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i}{T_j} \right\rceil C_j$$

- No es lineal ni continua
- No se puede resolver de manera analítica

- Solución iterativa basada en recurrencia:**

Dar valor inicial a w_i : $w_i^0 = C_i + \sum_{j \in hp(i)} C_j$

Hacer

$$w_i^{n+1} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_i^n}{T_j} \right\rceil C_j$$

Hasta que $w_i^{n+1} = w_i^n$ o bien $w_i^{n+1} > T_i$

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

15

Ejemplo

Task	T	C	P
τ_1	30	10	2
τ_2	20	7	3
τ_3	50	8	1

$$w_2^0 = C_i + \sum_{j \in hp(i)} C_j = 7$$

$$R_2 = w_2^0 \text{ (no hay tareas más prioritarias)}$$

$$w_1^0 = C_i + \sum_{j \in hp(i)} C_j = 10 + 7 = 17$$

$$w_1^1 = C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_1^0}{T_j} \right\rceil C_j = 10 + \left\lceil \frac{17}{20} \right\rceil 7 = 17 \rightarrow R_1 = 17$$

$$w_3^0 = C_i + \sum_{j \in hp(i)} C_j = 8 + 7 + 10 = 25$$

$$w_3^1 = C_i + \sum_{j \in hp(i)} \left\lceil \frac{w_3^0}{T_j} \right\rceil C_j = 8 + \left\lceil \frac{25}{20} \right\rceil 7 + \left\lceil \frac{25}{30} \right\rceil 10 = 8 + 14 + 10 = 32$$

$$w_3^2 = 8 + \left\lceil \frac{32}{20} \right\rceil 7 + \left\lceil \frac{32}{30} \right\rceil 10 = 8 + 14 + 20 = 42$$

$$w_3^3 = 8 + \left\lceil \frac{42}{20} \right\rceil 7 + \left\lceil \frac{42}{30} \right\rceil 10 = 8 + 21 + 20 = 49$$

$$w_3^4 = 8 + \left\lceil \frac{49}{20} \right\rceil 7 + \left\lceil \frac{49}{30} \right\rceil 10 = 8 + 21 + 20 = 49 \rightarrow R_3 = 49$$

- En este caso el factor de utilización no daba una respuesta
- Al calcular R_i tenemos una **condición necesaria y suficiente**

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

16

Modelo simplificado con tareas esporádicas

- **Tareas esporádicas:**
 - Las tareas esporádicas no tienen periodo, luego no se admitirían en el modelo simplificado
 - Para incorporarlas al modelo:
 - Considerar un pseudoperiodo: Menor intervalo posible entre activaciones
 - Aceptar que $D \leq T$ (no suele tener sentido en este caso que sean iguales)
- **Asignación óptima de prioridades: Monótona en plazo** (“deadline monotonic scheduling”)
 - Asignar mayor prioridad a las tareas con menor plazo
- **Verificación de los plazos:**
 - No es válido el criterio basado en factor de utilización
 - El cálculo del tiempo de respuesta sigue siendo válido, acabando si $w_i^{n+1} = w_i^n$ o $w_i^{n+1} > D_i$

02/12/2025

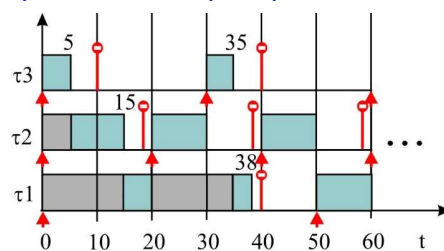
© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

17

Ejemplos

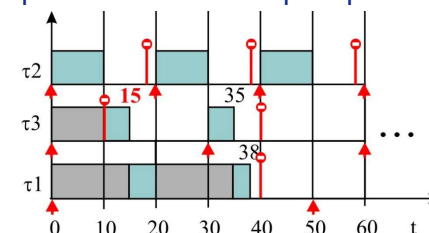
- Mayor prioridad a menor plazo: Se cumplen plazos

Task	T	C	D	P	R
τ_1	50	8	40	1	38
τ_2	20	10	18	2	15
τ_3	30	5	10	3	5



- Mayor prioridad a menor periodo: **NO** se cumplen plazos

Task	T	C	D	P	R
τ_1	50	8	40	1	38
τ_2	20	10	18	3	10
τ_3	30	5	10	2	15



02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

18

Interacción entre tareas

- **Interacción:**
 - El modelo simplificado supone tareas independientes
 - En la realidad interaccionan entre sí:
 - Memoria compartida
 - Paso de mensajes
- **Inversión de prioridad:**
 - Una tarea de alta prioridad ha de esperar a otra de baja prioridad
 - Sucede en memoria compartida (p.e. mutex) o paso de mensajes
 - El bloqueo en inversión de prioridad **no es deseable, pero a veces no puede evitarse**
 - Debe minimizarse y cuantificarse

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

19

Bloqueo en inversión de prioridad

La inversión de prioridad puede ser arbitrariamente larga:

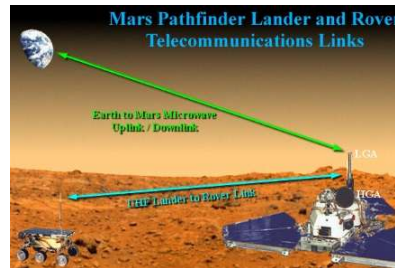
- Una tarea de baja prioridad T1 entra en su sección crítica asociada al recurso A
- Otra tarea de alta prioridad T2 también necesita A, luego ha de esperar a T1
- Otra tarea de prioridad intermedia, T3, no relacionada con T1 o T2, expulsa a T1
- Ahora T2 ha de esperar realmente a T3 (posiblemente durante mucho tiempo...)

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

20

Inversión de prioridad en Marte



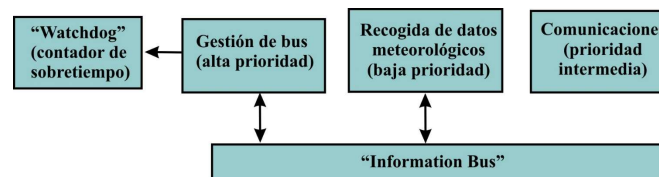
- El complejo aterrizaje de la misión Pathfinder fue un éxito
- El sistema se fue activando paso por paso
- Problema: **reinicio esporádico del sistema** con pérdida de datos
- Todo comenzó después de activar la tarea de recogida de datos meteorológicos (TRDM)

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

21

Inversión de prioridad en Marte



- La tarea TRDM cierra un "mutex" para publicar sus datos en el "Information Bus" (memoria compartida)
- La tarea de gestión también pide "mutex", pero se bloquea mientras no lo libere la tarea TRDM (normalmente poco tiempo)
- La tarea de comunicaciones expulsa a la tarea TRDM
- El "bus" deja de funcionar
- El temporizador de seguridad ("watchdog") reinicia el sistema
- **Solución:** Modificar el "mutex" para que funcionase con **protocolo de herencia de prioridad** (más seguro, aunque más costoso en tiempo)

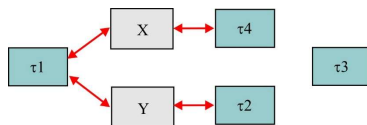
02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

22

Ejemplo

- Dos recursos compartidos: X e Y
- Cuatro tareas
 - La tarea 1 comparte X con la 4
 - La tarea 1 comparte Y con la 2
 - La tarea 3 es independiente



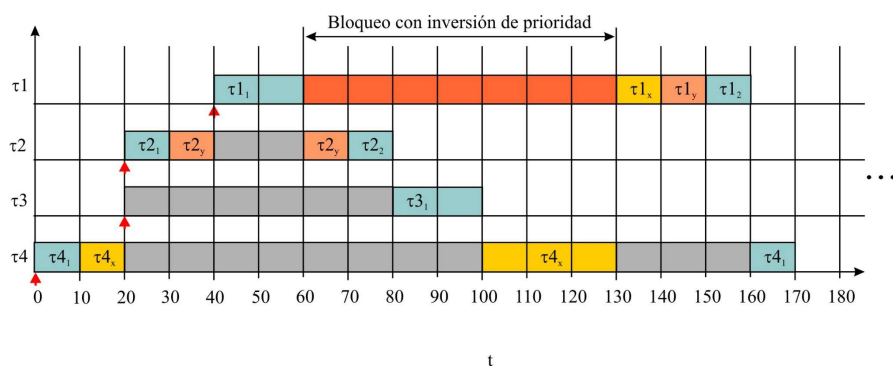
Tarea	P	Sección	Usa	C
τ1	4	τ1 ₁	-	20
		τ1 _x	X	10
		τ1 _y	Y	10
		τ1 ₂	-	10
τ2	3	τ2 ₁	-	10
		τ2 _y	Y	20
		τ2 ₂	-	10
τ3	2	τ3 ₁	-	20
τ4	1	τ4 ₁	-	10
		τ4 _x	X	40
		τ4 ₂	-	10

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

23

Inversión de prioridad



➡ El bloqueo dura tanto como tarden τ2 y τ3 en acabar

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

24

Herencia de prioridad

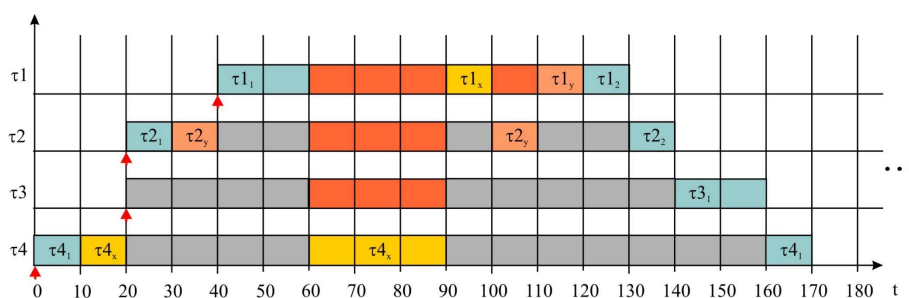
- Para reducir la duración del bloqueo:
 - Modificar dinámicamente la prioridad
 - La tarea bloqueada **transfiere** temporalmente su mayor prioridad a la tarea bloqueante (**herencia de prioridad**)
- Herencia de prioridad:
 - Más que herencia, es una transferencia de prioridad
 - Cada tarea tiene en cada instante como prioridad el máximo de dos valores:
 - Su prioridad básica
 - La prioridad de la tarea más prioritaria a la que está bloqueando
 - La herencia de prioridad es **transitiva**; si B bloquea a A, toma la prioridad de A; si C bloquea a B, también toma la prioridad de A
 - La inversión de prioridad se minimiza, y puede cuantificarse (se reduce a las secciones críticas)

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

25

Herencia de prioridad



Comentarios:

- La tarea τ_4 recibe la máxima prioridad de 60 a 90
- Una tarea puede bloquearse a causa de recursos que no usa (τ_2)
- Una tarea puede bloquearse aunque no utilice ningún recurso compartido (τ_3)
- La tarea de menor prioridad no se bloquea (aunque es expulsada)

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

26

Duración máxima del bloqueo

- Número máximo de bloqueos: Igual al de recursos compartidos que puedan causar bloqueo
- El máximo tiempo de bloqueo B_i es

$$B_i = \sum_{k=1}^K \text{uso}(k, i) C(k)$$

- K : Número de recursos
- $\text{uso}(k, i)$:
 - Vale 1 si el recurso k es utilizado al menos por una tarea de prioridad inferior a P_i y al menos por una tarea de prioridad igual o superior a P_i
 - Vale 0 en cualquier otro caso
- $C(k)$: Coste máximo de la sección crítica del recurso k
- Ecuación modificada para R_i :

$$R_i = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i}{T_j} \right\rceil C_j$$

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

27

Protocolos de techo de prioridad

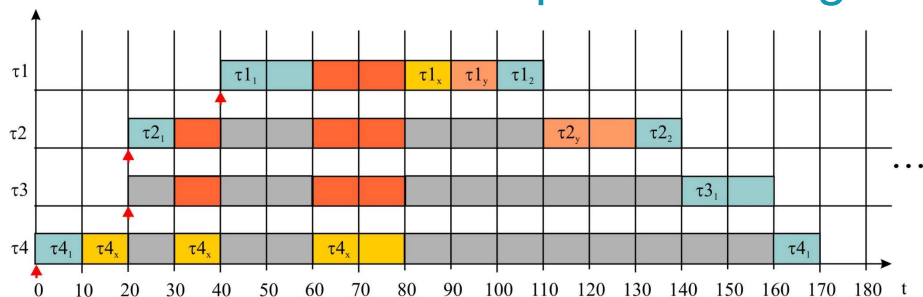
- Techo de prioridad de un recurso:
 - Máxima prioridad de las tareas que lo usan (valor estático)
- Protocolo de techo de prioridad original:
 - Como en el protocolo de herencia de prioridad, cada tarea tiene en cada instante como prioridad el máximo de dos valores:
 - Su prioridad básica
 - La prioridad de la tarea más prioritaria a la que está bloqueando
 - Una tarea sólo puede usar un recurso si su prioridad dinámica es **mayor que el techo de todos los recursos utilizados por otras tareas** (excluyendo los que ella esté ya utilizando)
- Razonamiento:
 - Se da acceso a un primer recurso con un determinado techo
 - Sólo puede accederse simultáneamente a un segundo recurso si no existen tareas de mayor prioridad que pudieran bloquearse a causa de ambos recursos
 - Una tarea sólo se bloquea con inversión de prioridad **una vez por ciclo**

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

28

Protocolo de techo de prioridad original



Comentarios:

- La tarea τ_4 es expulsada por τ_2 en 20
- La tarea τ_2 de prioridad 3 no puede usar el recurso Y, porque ya se está usando X y su techo es 4 – τ_2 **no supera el techo**
- La tarea τ_4 está ahora bloqueando a τ_2 , por lo que **su prioridad dinámica sube a 3** y puede expulsar a τ_3
- En 60 τ_1 no puede usar X, y es bloqueado. Ahora τ_4 **aumenta su prioridad dinámica a 4** y termina su sección crítica

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

29

Protocolo de techo de prioridad original

• Algunas propiedades:

- Una tarea determinada sólo se bloquea en inversión de prioridad una vez por ciclo
- No puede haber interbloqueo (“deadlock”)
- No puede haber bloqueos transitivos (B bloquea a A, y C bloquea a B, luego también indirectamente a A)
- La exclusión mutua se consigue sólo con el protocolo, sin necesidad de otros mecanismos adicionales

• Duración máxima del bloqueo:

- Como **sólo hay un bloqueo por ciclo**, cuenta el peor de los casos:

$$B_i = \max_{0 \leq k \leq K} [uso(k, i)C(k)]$$

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

30

Protocolo de techo de prioridad inmediato

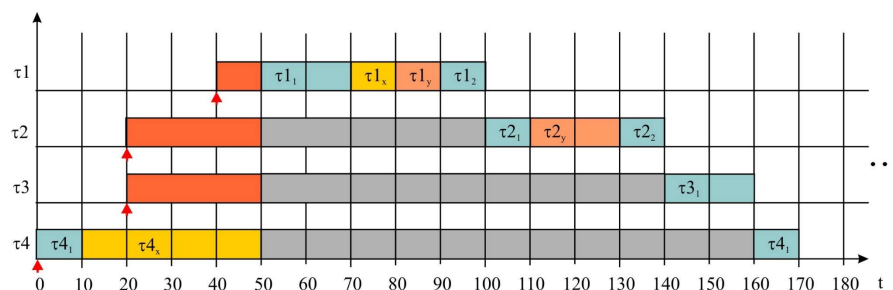
- **Definición:**
 - La prioridad dinámica de una tarea es el máximo de su prioridad básica y el mayor de los techos de prioridad de los recursos que está utilizando
- En este caso la tarea que accede a un recurso “hereda” (recibe) **inmediatamente** su techo de prioridad
- **Propiedades:**
 - Las mismas que el protocolo de techo de prioridad original
 - Además, **si una tarea se bloquea lo hace al principio de su ciclo**
- **Ventajas:**
 - Es más fácil de implementar que el original. No es necesario analizar los bloqueos existentes
 - Es más eficiente, en el sentido de generar menos cambios de contexto

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

31

Protocolo de techo de prioridad inmediato



Comentarios:

- La tarea τ_4 **adquiere prioridad dinámica 4** al usar el recurso X, por lo que ninguna otra tarea puede expulsarla
- Al acabar su sección crítica τ_4 **recupera su prioridad básica**
- La tarea τ_1 puede ahora expulsar a τ_4 y tiene ya todos los recursos disponibles

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

32

Planificación en POSIX

- Para **hilos y procesos** puede modificarse:
 - El algoritmo de planificación
 - La prioridad (y otros parámetros)
- No obstante, la unidad de planificación **es el hilo**
- **Ámbito de planificación (“scope”)**
 - Puede fijarse al crear el hilo
 - Planificación a nivel de sistema (PTHREAD_SCOPE_SYSTEM):
 - Los atributos de planificación del proceso no afectan al hilo
 - El hilo compite con el resto de hilos del sistema según sus propios atributos de planificación
 - Planificación a nivel de proceso (PTHREAD_SCOPE_PROCESS)
 - El hilo compite con el resto de hilos de su proceso según sus propios atributos de planificación
 - Los atributos de planificación del proceso se utilizan para competir a nivel de sistema, de un modo no completamente especificado

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

33

Planificación en POSIX

- El ámbito de planificación suele estar asociado a diferentes implementaciones de los hilos:
 - A nivel de usuario:
 - El concepto de hilo no existe a nivel del núcleo del sistema operativo (“kernel”). Sólo es posible la planificación a nivel de proceso
 - Habitualmente se planifica a dos niveles:
 - Un planificador a nivel de proceso utiliza los atributos de los hilos para escoger al que ha de ejecutarse de entre los pertenecientes a dicho proceso
 - Un planificador a nivel de sistema utiliza los atributos de los procesos para escoger al proceso que puede ejecutar su hilo
 - A nivel de “kernel”:
 - Los hilos son gestionados directamente por el núcleo del sistema operativo, de modo que son posibles ambos ámbitos de planificación
- **Asignación a procesadores (“allocation domain”)**
 - Importante; determina los recursos por los que hilos/procesos compiten
 - Depende de la implementación; no está normalizado

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

34

Planificación en POSIX

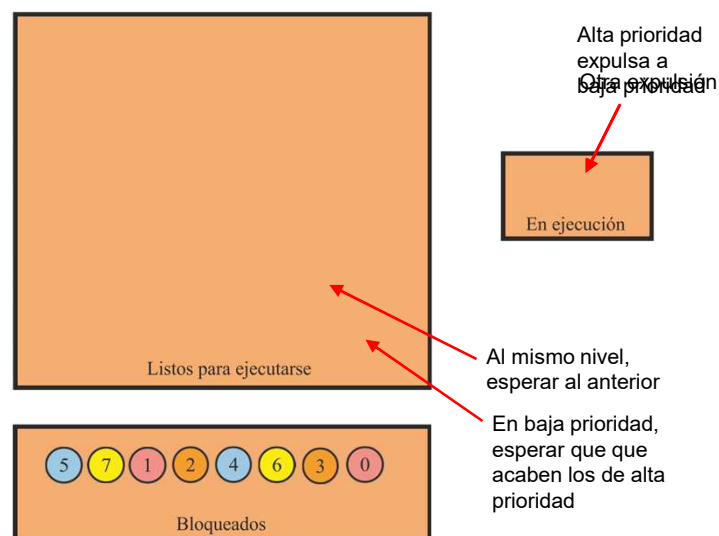
- Opciones normalizadas para el algoritmo de planificación:
 - SCHED_FIFO
 - SCHED_RR
 - SCHED_SPORADIC
 - SCHED_OTHER
- En cualquiera de ellas existe una **cola virtual de hilos en cada nivel de prioridad**
- **SCHED_FIFO:**
 - Los hilos que se desbloquean y pasan a estar listos para ejecutarse se ubican **al final** de la cola que les corresponde por su prioridad
 - Se escoge siempre al primer hilo de la cola de mayor prioridad
 - Los procesos/hilos de alta prioridad expulsan a los de baja prioridad, que pasan **al comienzo** de la cola de su nivel de prioridad
 - En el mismo nivel de prioridad los hilos **deben esperar a que los precedentes acaben o se bloqueen**

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

35

SCHED_FIFO



02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

36

Planificación en POSIX

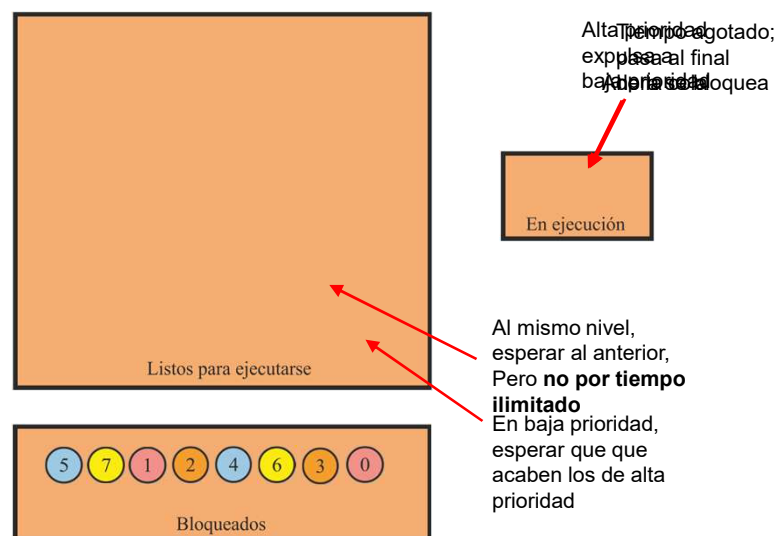
- **SCHED_RR** ("RR" de "round-robin"):
 - Igual que SCHED_FIFO, pero los hilos se ejecutan sólo durante un cierto intervalo de tiempo; después pasan al final de su cola
 - Los hilos **de un determinado nivel** "comparten" la máquina – ninguno puede acaparar la CPU a costa del resto
 - Un hilo **sí** que puede expulsar indefinidamente **a otros menos prioritarios**
 - Un hilo anteriormente expulsado que pasa de nuevo a ejecución puede completar el tiempo restante de su intervalo
- **SCHED_SPORADIC**:
 - Como SCHED_FIFO, pero los hilos se ejecutan a un nivel de prioridad alto hasta que se alcanza un límite de "capacidad" (tiempo de ejecución); después pasan a un nivel de prioridad inferior
 - La "capacidad" (tiempo de ejecución) se restaura al cabo de un cierto tiempo
 - Permite limitar el efecto de actividades esporádicas no críticas, evitando que sobrecarguen el sistema (a costa de su tiempo de respuesta)
- **SCHED_OTHER**:
 - Planificador no especificado; puede ser uno de los otros, o bien un planificador de propósito general
 - Significa que el hilo o proceso no necesita planificación en tiempo real

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

37

SCHED_RR



02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y Automática, ESI Sevilla)

38

Elegir planificación de procesos

- **Por defecto:**
 - Se hereda del proceso padre, incluso tras **exec**
 - **Dinámicamente:**
 - Un proceso puede cambiar política y parámetros de otro:
- int **sched_setscheduler** (pid_t pid, int policy,
struct sched_param *param);
- **policy:** Tipo de planificador (SCHED_FIFO, SCHED_RR...)
 - Estructura de tipo **sched_param**: parámetros del planificador, entre ellos la prioridad (**sched_priority**). Otros parámetros afectan a SCHED_SPORADIC

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

39

Elegir planificación para hilos

- **Por defecto**
 - Heredan la de aquel que los crea
 - El primer hilo de un proceso (**main**) tiene los atributos del proceso, que se heredan del proceso padre
- **Fijar planificación al crear el hilo**
 - Inicializar adecuadamente una estructura de atributos en la que se anule la “herencia” y se fijen política, parámetros y ámbito de planificación
 - Usarla en **pthread_create**
- **Cambiar dinámicamente la planificación**
 - Cualquier hilo puede cambiar la planificación de otro (o de sí mismo), salvo el ámbito de planificación, que ha de cambiarse en **pthread_create**:

```
int pthread_setschedparam(pthread_t hilo, int policy,  
struct sched_param *param);
```

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

40

Elegir planificación para hilos

```
(...)
#include <pthread.h>
void *hilo(void *p);
(...)
pthread_t idh; pthread_attr_t atr;
struct sched_param par; /* Parámetros de planificación (prioridad y otros) */

/* Preparar una estructura de atributos */
pthread_attr_init(&atr);
pthread_attr_setinheritsched(&atr, PTHREAD_EXPLICIT_SCHED); /* No heredarlos */
pthread_attr_setscope(&atr, PTHREAD_SCOPE_SYSTEM); /* Planif. en sistema */
pthread_attr_setschedpolicy(&atr, SCHED_FIFO); /* Política FIFO */
par.sched_priority = 20; /* Prioridad 20 */
pthread_attr_setschedparam(&atr, &par);

/* El hilo comienza a ejecutarse con esos atributos */
pthread_create(&idh, &atr, hilo, NULL);
(..)
/* Ahora cambiará a prioridad 10 */
par.sched_priority = 10;
pthread_setschedparam(idh, SCHED_FIFO, &par);
(...)
```

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

41

Mutex y semáforos en POSIX

- **Mutex y protocolos asociados a la inversión de prioridad:**
 - Es necesario modificar los atributos del “mutex” creando una estructura de atributos y usando **pthread_mutexattr_setprotocol**
 - Opciones:
 - PTHREAD_PRIO_INHERIT: Herencia de prioridad. El hilo que ha cerrado el “mutex” toma la prioridad del hilo más prioritario que está esperando por su causa
 - PTHREAD_PRIO_PROTECT: Techo de prioridad inmediato. Se asigna al “mutex” un valor de prioridad; cualquier hilo que cierre el “mutex” toma temporalmente la prioridad de éste
 - PTHREAD_PRIO_NONE: No hay herencia de prioridad (valor por defecto, pero no soportado en QNX...)
- **Semáforos POSIX:**
 - Si el algoritmo de planificación no es SCHED_OTHER, y está definido _POSIX_PRIORITY_SCHEDULING al activar el semáforo se desbloquea el hilo más prioritario, y a igualdad de prioridad el que lleve más tiempo esperando
 - La norma no define protección frente a inversión de prioridad

02/12/2025

© Joaquín Ferruz Melero 2025 (Dpto. Ing. Sist. y
Automática, ESI Sevilla)

42

Cambiar protocolo de mutex

```
(...)  
#include <pthread.h>  
(...)  
pthread_mutex_t mut;  
  
/* Preparar estructura de atributos para herencia de prioridad */  
/* Para techo de prioridad inmediato habría que fijar también el techo del mutex  
   con pthread_mutexattr_setprioceiling */  
  
pthread_mutexattr_t attr;  
pthread_mutexattr_init(&attr);  
pthread_mutexattr_setprotocol(&attr, PTHREAD_PRIO_INHERIT);  
  
/* Inicializar mutex con esos atributos */  
  
pthread_mutex_init(&mut, &attr);  
(..)
```